

0.9. Tout l'alphabet, avec une seule boucle

Le modulo donne la colonne, la division donne la ligne.

1. Ce qu'il faut comprendre

Ce qui est nouveau ici : `sizeof`, le modulo `%` et la division `/`. Le reste (`for`, `poser`, `ALPHABET[i]`) vient des chapitres précédents.

`sizeof(ALPHABET)` veut dire « la taille de ALPHABET » : le nombre de cases du tableau, **26**. On ne l'écrit plus à la main : si le tableau changeait de taille, la boucle suivrait toute seule. `i < sizeof(ALPHABET)` s'arrête donc après `i = 25`, la dernière lettre.

`i / 20` est une **division entière** : elle garde le quotient et **jette les virgules**. $5 / 20$ donne 0 (et non 0,25) ; $19 / 20$ donne 0 ; $20 / 20$ donne 1 ; $25 / 20$ donne 1. Tant que `i` est plus petit que 20, le résultat est 0 : c'est la **ligne 0**. À partir de 20, c'est 1 : la **ligne 1**.

`i % 20` se lit « i modulo 20 » : c'est le **reste** de cette même division. $5 \% 20$ donne 5 (20 ne rentre pas dans 5, tout reste) ; $19 \% 20$ donne 19 ; $20 \% 20$ donne 0 (la division tombe juste, il ne reste rien) ; $25 \% 20$ donne 5. Le reste monte de 0 à 19 puis **repart à 0** : c'est la **colonne**.

Imagine **20 places par rang**, comme au cinéma : `i / 20` dit combien de rangs **complets** sont déjà remplis (le rang de la lettre), `i % 20` dit combien de places il **reste** une fois ces rangs retirés (sa place dans le rang).

Tour par tour : **i = 0** → colonne 0, ligne 0, **A** · **i = 19** → colonne 19, ligne 0, **T** (la ligne est pleine) · **i = 20** → colonne **0**, ligne **1**, **U** (le reste repart à 0, le quotient passe à 1) · **i = 25** → colonne 5, ligne 1, **Z**.

C'est compliqué à lire : le chapitre suivant montre `poserS`, qui fait ce calcul à notre place.

2. Le programme, morceau par morceau

Le programme entier de la leçon, découpé comme l'œil le lit. Il fait **1352 octets** de cartouche.

1. LE POINT D'ENTRÉE : C'EST ICI QUE LA CONSOLE ARRIVE

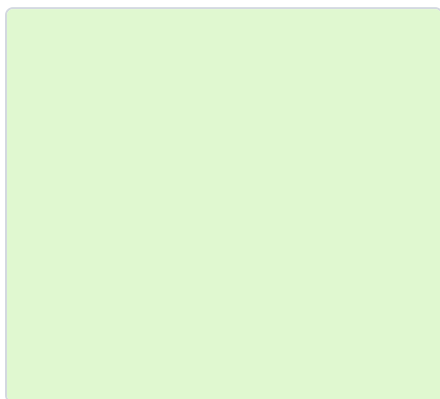
```
int main() {
    // for : répète le bloc entre { } ; i part de 0 et augmente de 1 à chaque tour.
    // sizeof(ALPHABET) = 26, la taille du tableau : 26 tours, i va de 0 à 25.
    for (uint8_t i = 0; i < sizeof(ALPHABET); i++) {

        // poser(colonne, ligne, tuile) :
        //  i % 20      = le RESTE de i divisé par 20 : 0, 1 ... 19, puis 0, 1 ... 5
        //              → la colonne, qui repart à 0 après la 19
        //  i / 20      = le QUOTIENT, sans virgule : 0 jusqu'à i = 19, puis 1
        //              → la ligne : 0 pour A à T, 1 pour U à Z
        //  ALPHABET[i] = la lettre n° i : A pour i = 0, Z pour i = 25
        poser(i % 20, i / 20, ALPHABET[i]);
    }

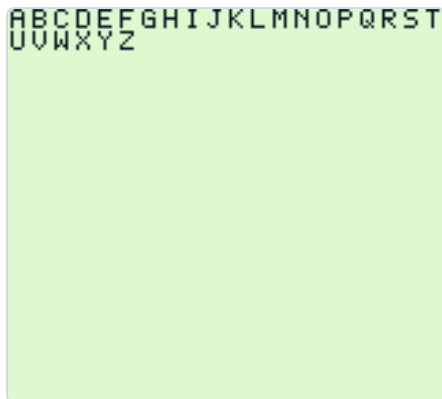
    while (true) {
        image();
    }
}
```

3. Ce qui se passe, étape par étape

La cartouche a été compilée et exécutée pour de bon ; voici son écran aux instants qui comptent. Une image identique à la précédente n'est pas montrée — un programme qui ne bouge pas n'a qu'une seule image, et c'est la vérité sur ce programme.



la toute première image — la console vient de démarrer — image 1



un tiers de seconde plus tard : le premier dessin est posé — image 20

4. Ce qu'on doit voir

A à T sur la ligne 0, U à Z en dessous, comme au 0.8.

5. Ce que la console répond

Ces contrôles ne sont pas décoratifs : ce sont ceux que `verifier-tuto.mjs` exige de cette leçon. Ils viennent d'être joués, et voici ce qu'ils ont mesuré.

✓ la ligne 0 : A à T

✓ la ligne 1 : U à Z

Ensuite : **0.9.1. Tout l'alphabet, avec une seule boucle — de base, ailleurs** — niveau 0 sur 10. · gameboy3 — écrire une cartouche Game Boy en C++